10 Most Common Reasons Kubernetes Deployments Fail (Part 1)

13 FEBRUARY 2017 on kubernetes, deployments

TWITTER FACEBOOK LINKEDIN HACKERNEWS POCKET

Over the last two years, I've worked with a number of teams to deploy their applications leveraging Kubernetes. Getting developers up to speed with Kubernetes jargon can be challenging, so when a Deployment fails, I'm usually paged to figure out what went wrong.

One of my primary goals when working with a client is to automate & educate myself out of that job, so I try to give developers the tools necessary to debug failed deployments. I've catalogued the most common reasons Kubernetes Deployments fail, and I'm sharing my troubleshooting playbook with you!

Without further ado, here are the 10 most common reasons Kubernetes Deployments fail:

1. Wrong Container Image / Invalid Registry Permissions

Two of the most common problems are (a) having the wrong container image specified and (b) trying to use private images without providing registry credentials. These are especially tricky when starting to work with Kubernetes or wiring up CI/CD for the first time.

Let's see an example. First, we'll create a deployment named `fail` pointing to a non-existent Docker image:

```
$ kubectl run fail --image=rosskukulinski/dne:v1.0.0
```

We can then inspect our Pods and see that we have one Pod with a status of `ErrImagePull` or `ImagePullBackOff`.

```
$ kubectl get pods
NAME                    READY     STATUS             RESTARTS   AGE
fail-1036623984-hxoas   0/1       ImagePullBackOff   0          2m
```

For some additional information, we can `describe` the failing Pod:

```
$ kubectl describe pod fail-1036623984-hxoas
```

If we look in the `Events` section of the output of the `describe` command we will see something like:

```
Events:
  FirstSeen    LastSeen    Count   From                        SubObjectPath
  ---------    --------    -----   ----                        -------------
  5m        5m      1   {default-scheduler }                        Norma
  5m        2m      5   {kubelet gke-nrhk-1-default-pool-c101b974-wfnZ} spec
```

```
5m        2m      5   {kubelet gke-nrhk-1-default-pool-a101b974-wfp7} spec
5m        2m      5   {kubelet gke-nrhk-1-default-pool-a101b974-wfp7} spec.
5m        2m      5   {kubelet gke-nrhk-1-default-pool-a101b974-wfp7}

5m    11s 19  {kubelet gke-nrhk-1-default-pool-a101b974-wfp7} spec.containe
5m    11s 19  {kubelet gke-nrhk-1-default-pool-a101b974-wfp7}
```

The error string, `Failed to pull image "rosskukulinski/dne:v1.0.0": Error: image rosskukulinski/dne not found` tells us that Kubernetes was not able to find the image `rosskukulinski/dne:v1.0.0`.

So then the question is: Why couldn't Kubernetes pull the image?

There are three primary culprits besides network connectivity issues:

- The image tag is incorrect
- The image doesn't exist (or is in a different registry)
- Kubernetes doesn't have permissions to pull that image

If you don't notice a typo in your image tag, then it's time to test using your local machine.

I usually start by running `docker pull` on my local development machine with the *exact same image tag*. In this case, I would run `docker pull rosskukulinski/dne:v1.0.0`.

- If this succeeds, then it probably means that Kubernetes doesn't have correct permissions to pull that image. Go read up on Image Pull Secrets to fix this issue.

- If the exact image tag fails, then I will test without an explicit image tag - `docker pull rosskukulinski/dne` - which will attempt to pull the `latest` tag. If this succeeds, then that means the original tag specified doesn't exist. This could be due to human error, typo, or maybe a misconfiguration of the CI/CD system.

If `docker pull rosskukulinski/dne` (*without an exact tag*) fails, then we have a bigger problem - that image does not exist at *all* in our image registry. By default, Kubernetes uses the Dockerhub registry. If you're using Quay.io, AWS ECR, or Google Container Registry, you'll need to specify the registry URL in the image string. For example, on Quay, the image would be `quay.io/rosskukulinski/dne:v1.0.0`.

If you *are* using Dockerhub, then you should double check the system that is publishing images to the registry. Make sure the name & tag match what your Deployment is trying to use.

Note: There is no observable difference in Pod status between a missing image and incorrect registry permissions. In either case, Kubernetes will report an `ErrImagePull` status for the Pods.

2. Application Crashing after Launch

Whether your launching a new application on Kubernetes or migrating an existing platform, having the application crash on startup is a common occurrence.

Let's create a new Deployment with an application that crashes after 1 second:

```
$ kubectl run crasher --image=rosskukulinski/crashing-app
```

Then let's take a look at the status of our Pods:

```
$ kubectl get pods
NAME                       READY     STATUS             RESTARTS   AGE
crasher-2443551393-vuehs   0/1       CrashLoopBackOff   2          54s
```

Ok, so `CrashLoopBackOff` tells us that Kuberenetes is trying to launch this Pod, but one or more of the containers is crashing or getting killed.

Let's `describe` the pod to get some more information:

```
$ kubectl describe pod crasher-2443551393-vuehs
Name:        crasher-2443551393-vuehs
Namespace:    fail
Node:        gke-nrhk-1-default-pool-a101b974-wfp7/10.142.0.2
Start Time:    Fri, 10 Feb 2017 14:20:29 -0500
Labels:        pod-template-hash=2443551393
        run=crasher
Status:        Running
IP:        10.0.0.74
Controllers:    ReplicaSet/crasher-2443551393
Containers:
  crasher:
    Container ID:    docker://51c940ab32016e6d6b5ed28075357661fef3282cb35691
    Image:        rosskukulinski/crashing-app
    Image ID:        docker://sha256:cf7452191b34d7797a07403d47a1ccf5254741d
    Port:
    State:        Terminated
      Reason:        Error
      Exit Code:    1
      Started:        Fri, 10 Feb 2017 14:22:24 -0500
      Finished:        Fri, 10 Feb 2017 14:22:26 -0500
```

```
    Last State:         Terminated
      Reason:         Error
      Exit Code:    1
      Started:        Fri, 10 Feb 2017 14:21:39 -0500
      Finished:         Fri, 10 Feb 2017 14:21:40 -0500
    Ready:          False
    Restart Count:    4
...
```

Awesome! Kubernetes is telling us that this Pod is being `Terminated` due to the application inside the container crashing. Specifically, we can see that the application `Exit Code` is `1`. We might also see an `OOMKilled` error, but we'll get to that later.

So our application is crashing ... why?

The first thing we can do is check our application logs. Assuming you are sending your application logs to `stdout` (which you should be!), you can see the application logs using `kubectl logs`.

```
$ kubectl logs crasher-2443551393-vuehs
```

Unfortunately, this Pod doesn't seem to have any log data. It's possible we're looking at a newly-restarted instance of the application, so we should check the previous container:

```
$ kubectl logs crasher-2443551393-vuehs --previous
```

Rats! Our application *still* isn't giving us anything to work with. It's probably time to add some additional log messages on startup to help debug the issue. We might also want to try

running the container locally to see if there are missing environmental variables or mounted volumes.

3. Missing ConfigMap or Secret

Kubernetes best practices recommend passing application run-time configuration via ConfigMaps or Secrets. This data could include database credentials, API endpoints, or other configuration flags.

A common mistake that I've seen developers make is to create Deployments that reference properties of ConfigMaps or Secrets that don't exist or even non-existent ConfigMaps/Secrets.

Let's see what that might look like.

Missing ConfigMap

For our first example, we're going to try to create a Pod that loads ConfigMap data as environmental variables.

```
# configmap-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: configmap-pod
spec:
  containers:
    - name: test-container
      image: gcr.io/google_containers/busybox
      command: [ "/bin/sh", "-c", "env" ]
      env:
        - name: SPECIAL_LEVEL_KEY
          valueFrom:
            configMapKeyRef:
              name: special-config
```

```
            key: special.how
```

Let's create a Pod, `kubectl create -f configmap-pod.yaml`. After waiting a few minutes, we can peek at our pods:

```
$ kubectl get pods
NAME            READY     STATUS              RESTARTS   AGE
configmap-pod   0/1       RunContainerError   0          3s
```

Our Pod's status says `RunContainerError`. We can use `kubectl describe` to learn more:

```
$ kubectl describe pod configmap-pod
[...]
Events:
  FirstSeen    LastSeen    Count   From                          SubObjectPath
  ---------    --------    -----   ----                          -------------
  20s        20s     1   {default-scheduler }
  19s        2s      3   {kubelet gke-ctm-1-sysdig2-35e99c16-tgfm}   spec.cor
  18s        2s      3   {kubelet gke-ctm-1-sysdig2-35e99c16-tgfm}   spec.cor
  18s        2s      3   {kubelet gke-ctm-1-sysdig2-35e99c16-tgfm}
```

The last item in the `Events` section explains what went wrong. The Pod is attempting to access a ConfigMap named `special-config`, but it's not found in this namespace. Once we create the ConfigMap, the Pod should restart and pull in the runtime data.

Accessing Secrets as environmental variables within your Pod specification will result in similar errors, like we've seen here with ConfigMaps.

But what if you're accessing a Secret or a ConfigMap via a volume?

Missing Secret

Here's a Pod spec that references a Secret named `myothersecret` and attempts to mount it as a volume.

```
# missing-secret.yaml
apiVersion: v1
kind: Pod
metadata:
  name: secret-pod
spec:
  containers:
    - name: test-container
      image: gcr.io/google_containers/busybox
      command: [ "/bin/sh", "-c", "env" ]
      volumeMounts:
        - mountPath: /etc/secret/
          name: myothersecret
  restartPolicy: Never
  volumes:
    - name: myothersecret
      secret:
        secretName: myothersecret
```

Let's create this Pod with `kubectl create -f missing-secret.yaml`.

After a few minutes, when we get our Pods, we'll see that it still is in the state of `ContainerCreating`.

```
$ kubectl get pods
```

```
NAME            READY     STATUS              RESTARTS   AGE
secret-pod      0/1       ContainerCreating   0          4h
```

That's odd ... let's `describe` the Pod to see whats going on.

```
$ kubectl describe pod secret-pod
Name:         secret-pod
Namespace:    fail
Node:         gke-ctm-1-sysdig2-35e99c16-tgfm/10.128.0.2
Start Time:   Sat, 11 Feb 2017 14:07:13 -0500
Labels:

Status:       Pending
IP:
Controllers:

[...]

Events:
  FirstSeen    LastSeen    Count   From                          SubObjectPath
  ---------    --------    -----   ----                          -------------
  18s          18s         1   {default-scheduler }                    Normal
  18s          2s          6   {kubelet gke-ctm-1-sysdig2-35e99c16-tgfm}
```

Once again, the `Events` section explains the problem. It's telling us that the Kubelet failed to mount a volume from the secret, `myothersecret`. To fix this problem, create `myothersecret` containing the necessary secure credentials. Once `myothersecret` has been created, the container will start correctly.

4. Liveness/Readiness Probe Failure

An important lesson for developers to learn when working with containers and Kubernetes is that just because your application

container is running, doesn't mean that it's *working*.

Kubernetes provides two essential features called Liveness Probes and Readiness Probes. Essentially, Liveness/Readiness Probes will periodically perform an action (e.g. make an HTTP request, open a tcp connection, or run a command in your container) to confirm that your application is working as intended.

If the Liveness Probe fails, Kubernetes will kill your container and create a new one. If the Readiness Probe fails, that Pod will not be available as a Service endpoint, meaning no traffic will be sent to that Pod until it becomes `Ready`.

If you attempt to deploy a change to your application that fails the Liveness/Readiness Probe, the rolling deploy will hang as it waits for all of your Pods to become Ready.

So what does this look like? Here's a Pod spec that defines a Liveness & Readiness Probe that checks for a healthy HTTP response for `/healthz` on port 8080.

```
apiVersion: v1
kind: Pod
metadata:
  name: liveness-pod
spec:
  containers:
    - name: test-container
      image: rosskukulinski/leaking-app
      livenessProbe:
        httpGet:
          path: /healthz
          port: 8080
```

```
        initialDelaySeconds: 3
        periodSeconds: 3
      readinessProbe:
        httpGet:
          path: /healthz
          port: 8080
        initialDelaySeconds: 3
        periodSeconds: 3
```

Let's create this Pod, `kubectl create -f liveness.yaml`, and then see what happens after a few minutes:

```
$ kubectl get pods
NAME           READY     STATUS    RESTARTS   AGE
liveness-pod   0/1       Running   4          2m
```

After 2 minutes, we can see that our Pod is still not "Ready", and it has been restarted four times. Let's `describe` the Pod for more information.

```
$ kubectl describe pod liveness-pod
Name:        liveness-pod
Namespace:    fail
Node:        gke-ctm-1-sysdig2-35e99c16-tgfm/10.128.0.2
Start Time:    Sat, 11 Feb 2017 14:32:36 -0500
Labels:
Status:        Running
IP:        10.108.88.40
Controllers:
Containers:
  test-container:
    Container ID:    docker://8fa6f99e6fda6e56221683249bae322ed864d686965dc4
    Image:        rosskukulinski/leaking-app
    Image ID:        docker://sha256:7bba8c34dad4ea155420f856cd8de37ba902604
    Port:
    State:        Running
      Started:        Sat, 11 Feb 2017 14:40:34 -0500
```

```
    Last State:         Terminated
      Reason:         Error
      Exit Code:    137
      Started:        Sat, 11 Feb 2017 14:37:10 -0500
      Finished:         Sat, 11 Feb 2017 14:37:45 -0500
[...]
Events:
  FirstSeen    LastSeen    Count   From                          SubObjectPath
  ---------    --------    -----   ----                          -------------
  8m        8m      1   {default-scheduler }                                 N
  8m        8m      1   {kubelet gke-ctm-1-sysdig2-35e99c16-tgfm}   spec.cont
  8m        8m      1   {kubelet gke-ctm-1-sysdig2-35e99c16-tgfm}   spec.cont
  7m        7m      1   {kubelet gke-ctm-1-sysdig2-35e99c16-tgfm}   spec.cont
  7m        7m      1   {kubelet gke-ctm-1-sysdig2-35e99c16-tgfm}   spec.cont
  8m    16s 10  {kubelet gke-ctm-1-sysdig2-35e99c16-tgfm}   spec.containers{t
  8m    1s  85  {kubelet gke-ctm-1-sysdig2-35e99c16-tgfm}   spec.containers{t
```

Once again, the `Events` section comes to the rescue. We can see that the Readiness and Liveness probes are both failing. The key string to look for is, `container "test-container" is unhealthy, it will be killed and re-created`. This tells us that Kubernetes is killing the container because the Liveness Probe has failed.

There are likely three possibilities:

1. Your Probes are now incorrect - Did the health URL change?
2. Your Probes are too sensitive - Does your application take a while to start or respond?
3. Your application is no longer responding correctly to the Probe - Is your database misconfigured?

Looking at the logs from your Pod is a good place to start debugging. Once you resolve this issue, a fresh Deployment should succeed.

5. Exceeding CPU/Memory Limits

Kubernetes gives cluster administrators the ability to limit the amount of CPU or memory allocated to Pods and Containers. As an application developer, you might not know about the limits and then be surprised when your Deployment fails.

Let's attempt to create this Deployment in a cluster with an unknown CPU/Memory request limit:

```
# gateway.yaml
apiVersion: extensions/v1beta1
kind: Deployment
metadata:
  name: gateway
spec:
  template:
    metadata:
      labels:
        app: gateway
    spec:
      containers:
        - name: test-container
          image: nginx
          resources:
            requests:
              memory: 5Gi
```

You'll notice that we're setting a resource request of 5Gi. Let's create the deployment: `kubectl create -f gateway.yaml`.

Now we can look at our Pods:

```
$ kubectl get pods
No resources found.
```

Huh? Let's inspect our Deployment using `describe`:

```
$ kubectl describe deployment/gateway
Name:            gateway
Namespace:        fail
CreationTimestamp:    Sat, 11 Feb 2017 15:03:34 -0500
Labels:            app=gateway
Selector:        app=gateway
Replicas:        0 updated | 1 total | 0 available | 1 unavailable
StrategyType:        RollingUpdate
MinReadySeconds:    0
RollingUpdateStrategy:    0 max unavailable, 1 max surge
OldReplicaSets:
NewReplicaSet:        gateway-764140025 (0/1 replicas created)

Events:
  FirstSeen    LastSeen    Count   From                SubObjectPath   Type
  ---------    --------    -----   ----                -------------   ------
  4m        4m      1   {deployment-controller }            Normal      Scali
```

Based on that last line, our deployment created a ReplicaSet (`gateway-764140025`) and scaled it up to 1. The ReplicaSet is the entity that manages the lifecycle of the Pods. We can `describe` the ReplicaSet:

```
$ kubectl describe rs/gateway-764140025
Name:        gateway-764140025
Namespace:    fail
Image(s):    nginx
Selector:    app=gateway,pod-template-hash=764140025
Labels:        app=gateway
        pod-template-hash=764140025
Replicas:    0 current / 1 desired
Pods Status:    0 Running / 0 Waiting / 0 Succeeded / 0 Failed
No volumes.
Events:
```

```
FirstSeen    LastSeen    Count   From                SubObjectPath   Type
---------    --------    -----   ----                -------------   ------
6m           28s         15      {replicaset-controller }            Warning     Faile
```

Ahh! There we go. The cluster administrator has set a maximum memory usage per Pod of `100Mi` (what a cheapskate!). You can inspect the current namespace limits by running `kubectl describe limitrange`.

You now now have three choices:

1. Ask your cluster admin to increase the limits
2. Reduce the Request or Limit settings for your Deployment
3. Go rogue and edit the limits (`kubectl edit` FTW!)

Check out Part 2!

And that's the first 5 most common reasons Kubernetes Deployments fail. Click here for Part 2 which has #6-10.

TWITTER FACEBOOK LINKEDIN HACKERNEWS POCKET

Ross Kukulinski

My name is Ross. I teach the world Kubernetes and Nodejs through consulting, conference speaking, and training courses.

Philadelphia, PA

READ THIS NEXT

10 Most Common Reasons Kubernetes Deployments Fail (Part 2)

Earlier this week I posted the first 5 most common reasons Kubernetes Deployments fail. Here is the remainder of...

Ross Kukulinski © 2022

YOU MIGHT ENJOY

Migrating from Heroku to Kubernetes

Originally posted on the ReactiveOps Blog Leveraging a Platform as a Service (PaaS) is a great way to quickly...